
Title: OASIS–JUMP Drug–Gene Morphological Matching: Exploratory Analysis

author: Dr. Parul Kulshreshtha

Introduction

Recently, Ewald et al. (2026) introduced OASIS, a large-scale Cell Painting resource generated in primary human hepatocytes to investigate whether image-based morphological profiling can improve the detection and mechanistic characterization of compound-induced toxicity. The study demonstrated that morphological profiles can predict a broad range of toxicological and biological endpoints, often providing information beyond conventional viability assays and supporting the use of Cell Painting as a rich phenotypic readout of cellular state.

A central premise of the OASIS study is that Cell Painting captures integrated cellular responses rather than direct biochemical target engagement. If morphology reflects downstream cellular programs and pathway-level perturbations, then compounds and genetic perturbations that induce similar cellular states may occupy nearby regions of a shared morphological feature space. This raises an intriguing question: can morphology-based profiling be used to connect drug-induced phenotypes with genetic perturbations and thereby generate mechanistic hypotheses about compound activity?

To explore this question, I reconstructed the published OASIS preprocessing workflow directly from the project repository by tracing the metadata generation, MAD normalization, and feature-selection procedures implemented in the source code. Using the released OASIS CellProfiler profiles, I then integrated morphology profiles from DILI-associated compounds with CRISPR knockout profiles from the JUMP Cell Painting consortium. Harmony batch correction was used to align perturbations originating from independent studies, followed by morphology-based matching between compounds and genetic perturbations across a shared feature space.

The objective of this analysis was not to reproduce the predictive modeling

experiments reported in the original manuscript, but rather to evaluate whether morphology-derived similarity could recover biologically meaningful relationships between chemical and genetic perturbations across independent Cell Painting datasets. More broadly, this work serves as an exploratory demonstration of how large-scale image-based profiling resources can be combined to generate hypotheses linking compound activity, cellular pathways, and gene function.

Reference

Ewald, J. D., Titterton, K. L., Bäuerle, A., Beatson, A., Boiko, D. A., Cabrera, Á. A., Cheah, J., Cimini, B., Gorissen, B. L., Harrill, J., Jones, T. R., Karczewski, K., Crute, C. E., Rouquie, D., Seal, S., Weisbart, E., White, B., Carpenter, A. E., & Singh, S. (2026). *Cell Painting for cytotoxicity and mode-of-action analysis in primary human hepatocytes*. *Cell Systems*, 17(5), 101566.
<https://doi.org/10.1016/j.cels.2026.101566>

Objective

This project aimed to reproduce the core OASIS preprocessing workflow directly from the published repository and extend the analysis beyond toxicity prediction by integrating OASIS drug-induced morphological profiles with JUMP CRISPR perturbation profiles. The goal was to investigate whether Cell Painting morphology can connect drug-induced phenotypes to genetic perturbations and generate mechanistic hypotheses.

What Was Done

- Downloaded the official OASIS CellProfiler profiles and metadata.
- Reconstructed the published preprocessing workflow by tracing the repository source code, including metadata generation, MAD normalization, feature selection, and consensus profile generation.
- Identified that InChI/InChIKey annotations were excluded from the released metadata and recovered chemical identifiers from the original Cell Painting Gallery metadata resources.
- Mapped OASIS DILI compounds to JUMP compounds using reconstructed chemical identifiers.
- Downloaded and processed 148 JUMP CRISPR plates containing 7,977 gene knockouts.
- Integrated OASIS and JUMP perturbations using PCA, Harmony batch

correction, and morphology-based similarity analysis.

Dataset Summary

Dataset	Size
OASIS CellProfiler profiles	21,455
Original morphology features	5,640
Features after preprocessing	833
JUMP CRISPR perturbation wells	51,185
Unique CRISPR knockouts	7,977
Final drug–gene similarity matrix	12 compounds × 7,977 genes

Results

Integration Assessment

To evaluate cross-source alignment, source-label silhouette scores were computed before and after Harmony correction. Harmony reduced the source silhouette score from **0.338** to **0.102**, indicating substantial reduction of source-specific structure while retaining some residual separation between compound and CRISPR datasets.

Top-Ranked Drug–Gene Relationships

The strongest morphology-based drug–gene relationships were:

Compound	Gene	Similarity	Rank
Panobinostat	PLK1	0.991	1 / 7,977
(E/Z)-Belinostat	MED17	0.927	31 / 7,977

These two pairs ranked within the top 5% of all candidate genes for their respective compounds.

A notable observation was that canonical biochemical targets (e.g., CYP51A1, PTGS1/PTGS2, ESR1, and HDAC family members) were generally not among the strongest morphological matches. Instead, the strongest similarities involved downstream cellular regulators, suggesting that Cell Painting captures integrated cellular responses and pathway-level effects rather than direct target

engagement.

Key Outcome

This project successfully:

- Reproduced the core OASIS preprocessing workflow directly from source code.
- Recovered metadata omitted from the released dataset.
- Built a cross-dataset framework linking OASIS drug perturbations with JUMP CRISPR perturbations.
- Generated morphology-based drug–gene relationships spanning 7,977 genetic perturbations.
- Extended OASIS into a new application area beyond toxicity prediction.

Limitations

- Only 12 DILI compounds were represented in the final matched dataset.
- OASIS and JUMP profiles were generated in different cell types and experimental systems.
- Morphological similarity should be interpreted as evidence of shared cellular states rather than direct molecular interactions.

Conclusion

This exploratory analysis demonstrates the feasibility of linking OASIS drug-induced morphological phenotypes to JUMP genetic perturbations through a shared Cell Painting feature space. By reconstructing the published OASIS preprocessing workflow and integrating it with JUMP CRISPR perturbation profiles, this project identified morphology-based relationships spanning 7,977 genetic perturbations. The strongest signals involved downstream cellular regulators rather than primary drug targets, supporting the broader conclusion that Cell Painting captures systems-level cellular responses and may provide a useful framework for mechanism-of-action discovery and biological hypothesis generation across independent perturbation datasets.

```
In [1]: # — Environment Setup —————  
  
import boto3  
from botocore import UNSIGNED  
from botocore.config import Config
```

```

import pandas as pd
import numpy as np
import os
import io
import json
import urllib.request
import warnings

warnings.filterwarnings('ignore')

# — Paths —————

BASE_DIR    = '/Users/dr.parul/Downloads/parul_carpenter_project'
OASIS_DIR   = f'{BASE_DIR}/data/oasis_compiled'
JUMP_DIR    = f'{BASE_DIR}/data/jump_profiles'
CRISPR_DIR  = f'{BASE_DIR}/data/jump_crispr'
FIGURES_DIR = f'{BASE_DIR}/figures'

for d in [OASIS_DIR, JUMP_DIR, CRISPR_DIR, FIGURES_DIR]:
    os.makedirs(d, exist_ok=True)

# — S3 connection —————

s3 = boto3.client(
    's3',
    config=Config(signature_version=UNSIGNED)
)

BUCKET = 'cellpainting-gallery'

print("✓ Environment initialized")
print(f"✓ OASIS_DIR: {OASIS_DIR}")
print(f"✓ JUMP_DIR: {JUMP_DIR}")
print(f"✓ CRISPR_DIR: {CRISPR_DIR}")

```

```

✓ Environment initialized
✓ OASIS_DIR: /Users/dr.parul/Downloads/parul_carpenter_project/data/oasis_compiled
✓ JUMP_DIR: /Users/dr.parul/Downloads/parul_carpenter_project/data/jump_profiles
✓ CRISPR_DIR: /Users/dr.parul/Downloads/parul_carpenter_project/data/jump_crispr

```

```
In [2]: print(pd.__version__)
        print(OASIS_DIR)
```

2.3.3

/Users/dr.parul/Downloads/parul_carpenter_project/data/oasis_compiled

```

import boto3 from botocore import UNSIGNED from botocore.config import
Config import pandas as pd import numpy as np import os import io import
urllib.request import warnings warnings.filterwarnings('ignore')

```

—— Paths

```
BASE_DIR = '/Users/dr.parul/Downloads/parul_carpenter_project' OASIS_DIR =
f'{BASE_DIR}/data/oasis_compiled' JUMP_DIR = f'{BASE_DIR}/data/jump_profiles'
CRISPR_DIR = f'{BASE_DIR}/data/jump_crispr' FIGURES_DIR =
f'{BASE_DIR}/figures'
```

```
for d in [OASIS_DIR, JUMP_DIR, CRISPR_DIR, FIGURES_DIR]: os.makedirs(d,
exist_ok=True)
```

—— S3 connection

```
s3 = boto3.client('s3', config=Config(signature_version=UNSIGNED)) BUCKET =
'cellpainting-gallery'
```

—— Download official OASIS compiled profiles from Zenodo ———

Source: Ewald et al. Cell Systems 2026 — Zenodo 17067683

```
ZENODO_BASE = 'https://zenodo.org/records/17067683/files'
```

```
files = { 'metadata.parquet': 0.7, 'cpcnn_raw.parquet': 46.0,
'cellprofiler_raw.parquet': 394.3, 'dino_raw.parquet': 381.4, }
```

```
print("Downloading official OASIS compiled profiles...") for filename, size_mb in
files.items(): local_path = f'{OASIS_DIR}/{filename}' if os.path.exists(local_path):
print(f" ✓ {filename}: cached") else: print(f" Downloading {filename}
({size_mb:.0f} MB)...") urllib.request.urlretrieve(f'{ZENODO_BASE}/{filename}',
local_path) print(f" ✓ {filename}: done")
```

```
print("\n✓ All files ready")
```

```
In [3]: import urllib.request
# Check actual repo structure via GitHub API
```

```
import json

url = 'https://api.github.com/repos/jessica-ewald/2024_09_09_Axiom_OASIS'
with urllib.request.urlopen(url) as r:
    contents = json.loads(r.read())

print("Root structure:")
for item in contents:
    print(f"    {item['type']}: {item['name']}")
```

Root structure:

- file: .gitignore
- dir: 0_prepare_data
- dir: 1_snakemake
- dir: 2_downstream_analysis
- file: LICENSE
- file: README.md
- file: install_env.sh
- file: pyproject.toml
- file: requirements.txt

```
In [4]: # Check each directory
for folder in ['0_prepare_data', '1_snakemake', '2_downstream_analysis']:
    url = f'https://api.github.com/repos/jessica-ewald/2024_09_09_Axiom_OASIS/{folder}'
    with urllib.request.urlopen(url) as r:
        contents = json.loads(r.read())
    print(f"\n{folder}/")
    for item in contents:
        print(f"    {item['type']}: {item['name']}")
```

```

0_prepare_data/
  file: 1A_download_metadata.py
  file: 1B_download_dino.py
  file: 1C_download_cpcnn.py
  file: 1D_download_cellprofiler.py
  file: 1E_download_image_index.py
  file: 2A_format_metadata.py
  file: 2B_format_dino.py
  file: 2C_format_cpcnn.py
  file: 2D_format_cellprofiler.py
  file: 2E_format_image_index.py
  file: 3A_download_invitrodb.sh
  file: 3B_extract_invitrodb.ipynb
  file: 4A_download_compiled_inputs.py

```

```

1_snakemake/
  file: .gitignore
  file: Snakefile
  dir: classifier
  dir: concresponse
  dir: inputs
  dir: preprocessing
  dir: pycytominer
  dir: rules
  dir: visualize

```

```

2_downstream_analysis/
  dir: compiled_results
  dir: manuscript_notebooks
  dir: other_notebooks

```

```

In [5]: # Check preprocessing and pycytominer folders
for folder in ['1_snakemake/preprocessing',
               '1_snakemake/pycytominer',
               '1_snakemake/rules']:
    url = f'https://api.github.com/repos/jessica-ewald/2024_09_09_Axio
    with urllib.request.urlopen(url) as r:
        contents = json.loads(r.read())
    print(f"\n{folder}/")
    for item in contents:
        print(f"  {item['type']}: {item['name']}")

```



```

1_snakemake/preprocessing/
  file: __init__.py
  file: feature_selection.py
  file: io.py
  file: metadata.py
  file: normalize.py
  file: outliers.py
  file: stats.py
  file: transform.py

```

```

1_snakemake/pycytominer/
  dir: operations

```

```

1_snakemake/rules/
  file: classifier.smk
  file: common.smk
  file: concresponse.smk
  file: processing.smk
  file: visualize.smk

```

```

In [6]: # Read the key preprocessing files
files_to_read = [
    '1_snakemake/preprocessing/normalize.py',
    '1_snakemake/preprocessing/feature_selection.py',
    '1_snakemake/rules/processing.smk',
]

for filepath in files_to_read:
    url = f'https://raw.githubusercontent.com/jessica-ewald/2024_09_09'
    try:
        with urllib.request.urlopen(url) as r:
            content = r.read().decode()
            print(f"\n=== {filepath} ===")
            print(content[:3000])
    except Exception as e:
        print(f"x {filepath}: {e}")

```

```

=== 1_snakemake/preprocessing/normalize.py ===
import numpy as np
import pandas as pd
import pycytominer

```

```

from preprocessing.io import merge_parquet, split_parquet

```

```

def mad(variant_feats_path, neg_stats_path, normalized_path):
    meta, vals, features = split_parquet(variant_feats_path)
    neg_stats = pd.read_parquet(neg_stats_path)
    neg_stats = neg_stats.query("feature in @features")

    # get counts and sort by plate
    plates, counts = np.unique(meta["Metadata_Plate"], return_counts=True)

```

```

    ix = np.argsort(meta["Metadata_Plate"])
    meta = meta.iloc[ix]
    vals = vals[ix]

    # get mad and median matrices for MAD normalization
    mads = neg_stats.pivot(index="Metadata_Plate", columns="feature", v
alues="mad")
    mads = mads.loc[plates, features].values
    medians = neg_stats.pivot(
        index="Metadata_Plate",
        columns="feature",
        values="median",
    )
    medians = medians.loc[plates, features].values

    # Get normalized features (epsilon = 0) for all plates that have MA
D stats
    # -= and /= are inplace operations. i.e save memory
    vals -= np.repeat(medians, counts, axis=0)
    vals /= np.repeat(mads, counts, axis=0)

    merge_parquet(meta, vals, features, normalized_path)

def spherize(input_path, normalized_path):
    dframe = pd.read_parquet(input_path)
    dframe = dframe.sample(frac=1.0)
    dframe = pycytominer.normalize(
        dframe,
        image_features=True,
        samples="Metadata_Compound == 'DMSO'",
        method="spherize",
    )
    dframe.to_parquet(normalized_path)

=== 1_snakemake/preprocessing/feature_selection.py ===
import logging

import pandas as pd
from pycytominer.operations import correlation_threshold, variance_thre
shold

from .metadata import find_feat_cols

logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)

def select_features(dframe_path, feat_thresh, feat_selected_path):
    '''Run feature selection'''

```

```

dframe = pd.read_parquet(dframe_path)
features = find_feat_cols(dframe.columns)

# Filter out features with low variance
low_variance = variance_threshold(dframe, features)
features = [f for f in features if f not in low_variance]
logger.info(f'{len(low_variance)} features removed by variance_threshold')

# Filter out features where any value exceeds threshold
too_large = [f for f in features if (dframe[f].abs() > feat_threshold).any()]
features = [f for f in features if f not in too_large]
logger.info(f'{len(too_large)} features removed due to large values')

# Removed highly correlated filters
high_corr = correlation_threshold(dframe, features)
features = [f for f in features if f not in high_corr]
logger.info(f'{len(high_corr)} features removed by correlation_threshold')

dframe.drop(columns=low_variance + high_corr, inplace=True)

dframe.reset_index(drop=True).to_parquet(feats_selected_path)

=== 1_snakemake/rules/processing.smk ===
wildcard_constraints:
    pipeline=r"[_a-zA-Z.\~0-9\~]*"

features = config["features"]
scenario = config["workflow"]
name = config["name"]

rule mad_normalize:
    input:
        f"outputs/{features}/{name}/profiles/variant_feats.parquet",
        f"outputs/{features}/{name}/profiles/neg_stats.parquet",
    output:
        f"outputs/{features}/{name}/profiles/mad.parquet",
    run:
        pp.normalize.mad(*input, *output)

rule int:
    input:
        f"outputs/{features}/{name}/profiles/{pipeline}.parquet",
    output:
        f"outputs/{features}/{name}/profiles/{pipeline}_int.parquet",
    run:
        pp.select_features(*input, *output)

```

```

rule featselect:
    input:
        f"outputs/{features}/{name}/profiles/{pipeline}.parquet",
    output:
        f"outputs/{features}/{name}/profiles/{pipeline}_featselect.pa
rquet",
    params:
        outlier_thresh=config["outlier_feat_thresh"],
    run:
        pp.select_features(*input, params.outlier_thresh, *output)

```

```

In [7]: # Install pycytominer – their exact tool
import subprocess
subprocess.run(['pip', 'install', 'pycytominer', '-q'], capture_output

from pycytominer.operations import correlation_threshold, variance_thr
print("✓ pycytominer installed")

# Load official OASIS compiled profiles
print("\nLoading official OASIS profiles...")

metadata = pd.read_parquet(f'{OASIS_DIR}/metadata.parquet')
cp_raw = pd.read_parquet(f'{OASIS_DIR}/cellprofiler_raw.parquet')

print(f"✓ Metadata: {metadata.shape}")
print(f"✓ CellProfiler raw: {cp_raw.shape}")
print(f"\nMetadata columns: {metadata.columns.tolist()[:10]}")
print(f"\nCP columns (first 5): {cp_raw.columns.tolist()[:5]}")
print(f"CP columns (last 5): {cp_raw.columns.tolist()[-5:]}")

```

✓ pycytominer installed

Loading official OASIS profiles...

✓ Metadata: (21456, 24)

✓ CellProfiler raw: (21455, 5671)

Metadata columns: ['Metadata_well_id', 'Metadata_source', 'Metadata_Plate', 'Metadata_Concentration', 'Metadata_Compound', 'Metadata_compound_target', 'Metadata_compound_pathway', 'Metadata_compound_research_area', 'Metadata_compound_clinical_information', 'Metadata_Well']

CP columns (first 5): ['Metadata_well_id', 'Metadata_source', 'Metadata_Plate', 'Metadata_Concentration', 'Metadata_Compound']

CP columns (last 5): ['Nuclei_Texture_Variance_RNA_3_03_256', 'Nuclei_Texture_Variance_RNA_5_00_256', 'Nuclei_Texture_Variance_RNA_5_01_256', 'Nuclei_Texture_Variance_RNA_5_02_256', 'Nuclei_Texture_Variance_RNA_5_03_256']

```

In [8]: # Understand the data structure
print("=== DATA STRUCTURE ===")
print(f"Metadata rows: {len(metadata)}")
print(f"CP rows: {len(cp_raw)}")

```

```

# Identify feature columns
meta_cols = [c for c in cp_raw.columns if c.startswith('Metadata_')]
feat_cols = [c for c in cp_raw.columns if not c.startswith('Metadata_')]
print(f"\nMetadata columns in CP: {len(meta_cols)}")
print(f"Feature columns: {len(feat_cols)}")

# Check compound and concentration structure
print(f"\nUnique compounds: {cp_raw['Metadata_Compound'].nunique()}")
print(f"Unique concentrations: {cp_raw['Metadata_Concentration'].nunique()}")
print(f"Concentrations: {sorted(cp_raw['Metadata_Concentration'].unique())}")

# Check DMSO controls
dms0 = cp_raw[cp_raw['Metadata_Compound'] == 'DMSO']
print(f"\nDMSO wells: {len(dms0)}")
print(f"Treated wells: {len(cp_raw) - len(dms0)}")

# Check missing values
missing = cp_raw[feat_cols].isnull().sum().sum()
print(f"\nTotal missing values: {missing:,}")
print(f"Features with missing: {(cp_raw[feat_cols].isnull().any()).sum()}")

```

=== DATA STRUCTURE ===

Metadata rows: 21456

CP rows: 21455

Metadata columns in CP: 31

Feature columns: 5640

Unique compounds: 1086

Unique concentrations: 17

Concentrations: [np.float64(0.0), np.float64(0.009128631092607975), np.float64(0.02738589234650135), np.float64(0.0456431545317173), np.float64(0.0821576789021492), np.float64(0.1369294673204422), np.float64(0.2000000298023224), np.float64(0.4107883870601654), np.float64(0.699999998079071), np.float64(1.0), np.float64(2.200000047683716), np.float64(3.5), np.float64(6.699999809265137), np.float64(11.0), np.float64(20.0), np.float64(33.5), np.float64(100.0)]

DMSO wells: 4849

Treated wells: 16606

Total missing values: 0

Features with missing: 0

```

In [9]: # — Section 3: Preprocessing Pipeline —————
# Following exact pipeline from:
# jessica-ewald/2024_09_09_Axiom_OASIS/1_snakemake/preprocessing/
# normalize.py → mad normalization
# feature_selection.py → variance, outlier, correlation thresholds

# Reconstruct key preprocessing steps described in the OASIS workflow
# based on normalize.py, feature_selection.py, and processing.smk

```

```

# 1. MAD normalization per plate using DMSO controls
# 2. Variance threshold
# 3. Outlier threshold
# 4. Correlation threshold

print("Step 1: MAD normalization (DMSO per plate)...")

normalized = cp_raw.copy()
plates = cp_raw['Metadata_Plate'].unique()

for plate in plates:
    plate_mask = cp_raw['Metadata_Plate'] == plate
    dms_mask = plate_mask & (cp_raw['Metadata_Compound'] == 'DMSO')

    if dms_mask.sum() < 3:
        continue

    dms_vals = cp_raw.loc[dms_mask, feat_cols]
    medians = dms_vals.median()
    # MAD = median absolute deviation
    mads = (dms_vals - medians).abs().median()
    mads = mads.replace(0, np.nan)

    normalized.loc[plate_mask, feat_cols] = (
        cp_raw.loc[plate_mask, feat_cols] - medians
    ) / mads

# Drop features with NaN
nan_features = normalized[feat_cols].isnull().any()
features_to_drop = nan_features[nan_features].index.tolist()
normalized = normalized.drop(columns=features_to_drop)
feat_cols_norm = [c for c in feat_cols if c not in features_to_drop]
print(f"✓ MAD normalized: {len(feat_cols_norm)} features remaining "
      f"({len(features_to_drop)} dropped)")

# Step 2: Variance threshold
print("\nStep 2: Variance threshold...")
low_var = variance_threshold(normalized, feat_cols_norm)
feat_cols_norm = [f for f in feat_cols_norm if f not in low_var]
print(f"✓ {len(low_var)} removed, {len(feat_cols_norm)} remaining")

# Step 3: Outlier threshold
print("\nStep 3: Outlier threshold (>500)...")
too_large = [f for f in feat_cols_norm
              if (normalized[f].abs() > 500).any()]
feat_cols_norm = [f for f in feat_cols_norm if f not in too_large]
normalized = normalized.drop(columns=too_large)
print(f"✓ {len(too_large)} removed, {len(feat_cols_norm)} remaining")

# Step 4: Correlation threshold
print("\nStep 4: Correlation threshold...")
high_corr = correlation_threshold(normalized, feat_cols_norm)

```

```

feat_cols_norm = [f for f in feat_cols_norm if f not in high_corr]
normalized = normalized.drop(columns=high_corr)
print(f"✓ {len(high_corr)} removed, {len(feat_cols_norm)} remaining")

print(f"\n✓ Final features: {len(feat_cols_norm)}")
print(f"✓ Final shape: {normalized.shape}")

```

Step 1: MAD normalization (DMSO per plate)...

✓ MAD normalized: 5506 features remaining (134 dropped)

Step 2: Variance threshold...

✓ 1 removed, 5505 remaining

Step 3: Outlier threshold (>500)...

✓ 734 removed, 4771 remaining

Step 4: Correlation threshold...

✓ 3938 removed, 833 remaining

✓ Final features: 833

✓ Final shape: (21455, 865)

In [10]: *# Validate coverage of OASIS DILI-positive compounds in the compiled d*
from rdkit.Chem.inchi **import** InchiToInchiKey

DILI positive compounds from OASIS work

```

DILI_POSITIVE = [
    'Panobinostat', 'Romidepsin', '(E/Z)-Belinostat', 'Tacrolimus',
    'Troglitazone', 'Diclofenac', 'Isoniazid', 'Tamoxifen',
    'Valproic acid', 'Ketoconazole', 'Clarithromycin', 'Erythromycin',
    'Rifampicin', 'Fluconazole', 'Ibuprofen', 'Acetaminophen',
    'Tetracycline', 'Chlorpromazine', 'Nitrofurantoin', 'Sulindac',
    'Trovaflaxacin', 'Nefazodone', 'Erdafitinib', 'Famoxadone',
    'Indocyanine green'
]

```

Check which DILI compounds are in official OASIS profiles

```

print("=== DILI COMPOUNDS IN OFFICIAL OASIS PROFILES ===")
oasis_compounds = normalized['Metadata_Compound'].unique()
found_dili = [c for c in DILI_POSITIVE if c in oasis_compounds]
missing_dili = [c for c in DILI_POSITIVE if c not in oasis_compounds]

print(f"Found: {len(found_dili)}/25")
print(f"\nFound compounds:")
for c in found_dili:
    print(f"  ✓ {c}")
print(f"\nMissing compounds:")
for c in missing_dili:
    print(f"  x {c}")

```

=== DILI COMPOUNDS IN OFFICIAL OASIS PROFILES ===

Found: 25/25

Found compounds:

- ✓ Panobinostat
- ✓ Romidepsin
- ✓ (E/Z)-Belinostat
- ✓ Tacrolimus
- ✓ Troglitazone
- ✓ Diclofenac
- ✓ Isoniazid
- ✓ Tamoxifen
- ✓ Valproic acid
- ✓ Ketoconazole
- ✓ Clarithromycin
- ✓ Erythromycin
- ✓ Rifampicin
- ✓ Fluconazole
- ✓ Ibuprofen
- ✓ Acetaminophen
- ✓ Tetracycline
- ✓ Chlorpromazine
- ✓ Nitrofurantoin
- ✓ Sulindac
- ✓ Trovafloxacin
- ✓ Nefazodone
- ✓ Erdafitinib
- ✓ Famoxadone
- ✓ Indocyanine green

Missing compounds:

```
In [11]: from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics.pairwise import cosine_similarity
import harmony as hm

# Get DILI compound profiles – all concentrations
dili_profiles = normalized[
    normalized['Metadata_Compound'].isin(DILI_POSITIVE)
].copy()

print(f"DILI compound wells: {len(dili_profiles)}")
print(f"Unique compounds: {dili_profiles['Metadata_Compound'].nunique()}")
print(f"Concentrations per compound (median): ")
    f"{dili_profiles.groupby('Metadata_Compound').size().median():.0f}"

# Following the Ewald et al. consensus-profile strategy:
# median aggregation across all available concentrations
print("\nComputing consensus profiles (median across all concentration")
consensus = dili_profiles.groupby('Metadata_Compound')[feat_cols_norm]
print(f"✓ Consensus profiles: {consensus.shape}")
print(f" {consensus.shape[0]} compounds × {consensus.shape[1]} features")
```


DILI compound wells: 386

Unique compounds: 25

Concentrations per compound (median): 16

Computing consensus profiles (median across all concentrations)...

✓ Consensus profiles: (25, 833)

25 compounds × 833 features

Metadata reconstruction Initial attempts to map OASIS compounds to JUMP using the released metadata revealed that InChI/InChIKey annotations were not included in the published OASIS metadata. Inspection of the metadata-generation workflow (2A_format_metadata.py) confirmed that these fields were intentionally excluded. To enable chemical matching, compound annotations were recovered from the original OASIS biochem metadata files hosted on Cell Painting Gallery S3, from which InChIKeys were reconstructed and used for OASIS-JUMP compound mapping.

```
In [12]: # Inspect OASIS metadata fields
print("Metadata columns:")
print(metadata.columns.tolist())
print("\nSample rows for a DILI compound:")
print(metadata[metadata['Metadata_Compound'] == 'Panobinostat'].head(2)
```

Metadata columns:

```
['Metadata_well_id', 'Metadata_source', 'Metadata_Plate', 'Metadata_Concentration', 'Metadata_Compound', 'Metadata_compound_target', 'Metadata_compound_pathway', 'Metadata_compound_research_area', 'Metadata_compound_clinical_information', 'Metadata_Well', 'Metadata_row', 'Metadata_col', 'Metadata_mtt_lumi', 'Metadata_ldh_abs_signal', 'Metadata_ldh_abs_background', 'Metadata_ldh_abs', 'Metadata_mtt_normalized', 'Metadata_ldh_normalized', 'Metadata_mtt_ridge_norm', 'Metadata_ldh_ridge_norm', 'Metadata_OASIS_ID', 'Metadata_Perturbation', 'Metadata_Log10Conc', 'Metadata_Count_Cells']
```

Sample rows for a DILI compound:

Metadata_well_id	Metadata_source	Metadata_Plate	Metadata_Concentration	Metadata_Compound	Metadata_compound_target	Metadata_compound_pathway	Metadata_compound_research_area	Metadata_compound_clinical_information	Metadata_Well	Metadata_row	Metadata_col	Metadata_mtt_lumi	Metadata_ldh_abs_signal	Metadata_ldh_abs_background	Metadata_ldh_abs	Metadata_mtt_normalized	Metadata_ldh_normalized	Metadata_mtt_ridge_norm	Metadata_ldh_ridge_norm	Metadata_OASIS_ID	Metadata_Perturbation	Metadata_Log10Conc	Metadata_Count_Cells
9658	assayworks_prod_25_p=plate_41002692_r=10_c=21	assayworks_prod_25_plate_41002692	100.000000	Panobinostat	Apoptosis; Autophagy; HDAC; HIV	Anti-infection; Apoptosis; Autophagy; Cell Cycle/DNA Damage; Epigenetics																	
0.506			0.021																				
102822			0.313332																				
0.506674		OASIS1808		Panobinostat_100.0																			
3.817746			45																				
9659	assayworks_prod_26_p=plate_41002887_r=8_c=9	assayworks_prod_26_plate_41002887	0.410788	Panobinostat	Apoptosis; Autophagy; HDAC; HIV	Anti-infection; Apoptosis; Autophagy; Cell Cycle/DNA Damage; Epigenetics																	
0.448			0.020																				
696508			0.008572																				
0.054899		OASIS1808		Panobinostat_0.4107883870601654																			
1.431364			633																				

```
In [13]: # Check format_metadata.py
url = 'https://raw.githubusercontent.com/jessica-ewald/2024_09_09_Axio
with urllib.request.urlopen(url) as r:
    print(r.read().decode())
```

```
import os
```

```
import numpy as np
import polars as pl
from tqdm import tqdm
```

```
meta_keep = [
    "well_id",
```

```

        "source",
        "plate",
        "compound_concentration_um",
        "compound_name",
        "compound_target",
        "compound_pathway",
        "compound_research_area",
        "compound_clinical_information",
        "compound_url",
        "compound_scode",
        "well",
        "row",
        "col",
        "mtt_lumi",
        "ldh_abs_signal",
        "ldh_abs_background",
        "ldh_abs",
        "mtt_normalized",
        "ldh_normalized",
        "mtt_ridge_norm",
        "ldh_ridge_norm",
        "OASIS_ID",
    ]

```

```

def process_meta(input_meta_path: str, meta_nms: list) -> pl.DataFrame:
    """Process metadata.

```

Select columns of interest and rename.

```

    """
    meta = pl.read_parquet(input_meta_path)
    missing_cols = [i for i in meta_keep if i not in meta.columns]
    for mc in missing_cols:
        meta = meta.with_columns(
            pl.lit(None).alias(mc),
        )
    meta = meta.select(meta_keep)
    meta.columns = meta_nms

    return meta

```

```

def main() -> None:
    """Format metadata.

    Merge metadata from each plate into one file.

    """
    # Process metadata

```

```

meta_nms = [f"Metadata_{i}" for i in meta_keep]
meta_path = "../1_snakemake/inputs/metadata/biochem"
cc_path = "../1_snakemake/inputs/metadata/cc.parquet"
meta = []

plates = os.listdir(meta_path)
for plate in tqdm(plates):
    plate_path = f"{meta_path}/{plate}"
    meta.append(process_meta(plate_path, meta_nms))

meta = pl.concat(meta, how="vertical_relaxed")
meta = meta.rename({
    "Metadata_plate": "Metadata_Plate",
    "Metadata_well": "Metadata_Well",
    "Metadata_compound_name": "Metadata_Compound",
    "Metadata_compound_concentration_um": "Metadata_Concentration",
}).with_columns(
    pl.concat_str(["Metadata_Compound", "Metadata_Concentration"],
separator="_").alias("Metadata_Perturbation"),
    pl.col("Metadata_Well")
    .map_elements(lambda well: well if len(well) == 3 else f"{well[0]}{well[1]}", return_dtype=pl.Utf8)
    .alias("Metadata_Well"),
)

compounds = meta.select("Metadata_Compound").to_series().unique().to_list()
compounds = [i for i in compounds if i != "DMSO"]
meta_log10 = meta.filter(pl.col("Metadata_Compound") == "DMSO").with_columns(
    pl.lit(0).cast(pl.Float64).alias("Metadata_Log10Conc"),
)

for compound in compounds:
    temp = meta.filter(pl.col("Metadata_Compound") == compound)
    concs = temp.select(pl.col("Metadata_Concentration")).to_series().sort().unique().to_list()
    shift_val = np.abs(np.log10(concs[0] / 3))

    temp = temp.with_columns(
        (pl.col("Metadata_Concentration").log10() + shift_val).alias("Metadata_Log10Conc"),
    )

    meta_log10 = pl.concat([meta_log10, temp], how="vertical")

cc = pl.read_parquet(cc_path)
meta_log10 = meta_log10.join(cc, on=["Metadata_Plate", "Metadata_Well"])

# Filter out plates with different microscope
meta_log10 = meta_log10.filter(

```

```

        ~pl.col("Metadata_Plate").is_in(["plate_41002698", "plate_41002
695", "plate_41002696"])),
    )

    meta_log10.write_parquet("../1_snakemake/inputs/metadata/metadata.p
arquet")

if __name__ == "__main__":
    main()

```

```

In [18]: # Load JUMP compound metadata
import urllib.request

METADATA_BASE = 'https://raw.githubusercontent.com/jump-cellpainting/d

with urllib.request.urlopen(f'{METADATA_BASE}/compound.csv.gz') as r:
    jump_compounds = pd.read_csv(r, compression='gzip')
    jump_compounds = jump_compounds.dropna(subset=['Metadata_InChIKey'])

print(jump_compounds.shape)
print(jump_compounds.columns.tolist())

```

```

(115795, 4)
['Metadata_JCP2022', 'Metadata_InChIKey', 'Metadata_InChI', 'Metadata_S
MILES']

```

```

In [19]: print('jump_compounds' in globals())

```

```
True
```

```

In [20]: # The released metadata does not retain InChI/InChIKey fields
# Checked: jessica-ewald/2024_09_09_Axiom_OASIS/0_prepare_data/2A_form
# The released metadata does not retain InChI/InChIKey fields
# Only compound_name, target, pathway, research_area are kept
# Therefore: InChI retrieved from raw biochem parquet files on S3
# which contain the full compound annotation including InChI strings
# This is consistent with the OASIS compound list at oasisconsortium.o

print("Note: InChI not in official OASIS metadata (by design)")
print("Retrieving InChI from raw S3 biochem files...\n")

dili_inchi_map = {}
for batch in ['prod_25', 'prod_26', 'prod_27', 'prod_30']:
    response = s3.list_objects_v2(
        Bucket='cellpainting-gallery',
        Prefix=f'cpg0037-oasis/axiom/workspace/metadata/{batch}/',
    )
    for obj in response.get('Contents', []):
        if 'biochem.parquet' not in obj['Key']:
            continue
        data_obj = s3.get_object(Bucket='cellpainting-gallery',

```

```

Key=obj['Key'])
df = pd.read_parquet(io.BytesIO(data_obj['Body'].read()))
dili_rows = df[
    df['compound_name'].isin(DILI_POSITIVE) &
    df['compound_inchi'].notna()
][['compound_name', 'compound_inchi']].drop_duplicates()
for _, row in dili_rows.iterrows():
    if row['compound_name'] not in dili_inchi_map:
        dili_inchi_map[row['compound_name']] = row['compound_i

print(f"✓ InChI retrieved: {len(dili_inchi_map)}/25 compounds")

# Compute InChIKeys via RDKit
dili_inchikey_map = {}
for name, inchi in dili_inchi_map.items():
    key = InchiToInchiKey(inchi)
    if key:
        dili_inchikey_map[name] = key
print(f"✓ InChIKeys computed: {len(dili_inchikey_map)}/25")

# Match to JUMP compound library
matched = []
for name, inchikey in dili_inchikey_map.items():
    exact = jump_compounds[
        jump_compounds['Metadata_InChIKey'] == inchikey
    ]
    if len(exact) > 0:
        for _, row in exact.iterrows():
            matched.append({'compound_name': name,
                           'Metadata_JCP2022': row['Metadata_JCP2022'],
                           'match_type': 'exact'})
    else:
        first_block = inchikey.split('-')[0]
        partial = jump_compounds[
            jump_compounds['Metadata_InChIKey'].str.startswith(
                first_block, na=False)
        ]
        if len(partial) > 0:
            for _, row in partial.iterrows():
                matched.append({'compound_name': name,
                               'Metadata_JCP2022': row['Metadata_JCP20
                               'match_type': 'partial'})
        else:
            print(f" x No JUMP match: {name}")

matched_df = pd.DataFrame(matched)
print(f"\n✓ Matched in JUMP: {matched_df['compound_name'].nunique()}/2
print(f" Exact: {(matched_df['match_type']=='exact').sum()}")
print(f" Partial: {(matched_df['match_type']=='partial').sum()}")

```

Note: InChI not in official OASIS metadata (by design)
Retrieving InChI from raw S3 biochem files...

- ✓ InChI retrieved: 25/25 compounds
- ✓ InChIKeys computed: 25/25
 - x No JUMP match: Troglitazone
 - x No JUMP match: Rifampicin
 - x No JUMP match: Famoxadone
 - x No JUMP match: Romidepsin
 - x No JUMP match: Acetaminophen
 - x No JUMP match: Nitrofurantoin
 - x No JUMP match: Tacrolimus
 - x No JUMP match: Indocyanine green
 - x No JUMP match: Tetracycline
- ✓ Matched in JUMP: 16/25
 - Exact: 8
 - Partial: 8

```
In [21]: #checking cache
print(os.path.exists(JUMP_DIR))
print(os.path.exists(CRISPR_DIR))
print(len(os.listdir(JUMP_DIR)))
print(len(os.listdir(CRISPR_DIR)))
```

```
True
True
10
148
```

```
In [22]: # Check what's already cached locally
import os

compound_cached = os.listdir(JUMP_DIR)
crispr_cached = os.listdir(CRISPR_DIR)

print(f"Compound plates already cached: {len(compound_cached)}")
print(f"CRISPR plates already cached: {len(crispr_cached)}")

# Check total size
compound_size = sum(os.path.getsize(f'{JUMP_DIR}/{f}')
                    for f in compound_cached) / 1024/1024
crispr_size = sum(os.path.getsize(f'{CRISPR_DIR}/{f}')
                  for f in crispr_cached) / 1024/1024

print(f"\nCompound data: {compound_size:.0f} MB")
print(f"CRISPR data: {crispr_size:.0f} MB")
print(f"Total cached: {compound_size + crispr_size:.0f} MB")

# How many CRISPR plates still needed?
total_crispr = 148
still_needed = total_crispr - len(crispr_cached)
still_needed_mb = still_needed * 12.9
```

```
print(f"\nCRISPR plates still needed: {still_needed}")
print(f"Still to download: {still_needed_mb:.0f} MB")
```

Compound plates already cached: 10
CRISPR plates already cached: 148

Compound data: 129 MB
CRISPR data: 1914 MB
Total cached: 2043 MB

CRISPR plates still needed: 0
Still to download: 0 MB

In [24]: *# Load JUMP plate metadata*

```
METADATA_BASE = 'https://raw.githubusercontent.com/jump-cellpainting/d
with urllib.request.urlopen(f'{METADATA_BASE}/plate.csv.gz') as r:
    jump_plate = pd.read_csv(r, compression='gzip')

print(jump_plate.shape)
print(jump_plate.columns.tolist())
```

(2525, 4)
['Metadata_Source', 'Metadata_Batch', 'Metadata_Plate', 'Metadata_Plate Type']

In [25]: *# Download all remaining CRISPR plates*

```
crispr_plate_list = jump_plate[
    (jump_plate['Metadata_Source'] == 'source_13') &
    (jump_plate['Metadata_PlateType'] == 'CRISPR')
][['Metadata_Plate', 'Metadata_Batch']].values.tolist()

print(f"Total CRISPR plates: {len(crispr_plate_list)}")
print(f"Already cached: {len(crispr_cached)}")
print(f"To download: {len(crispr_plate_list) - len([p for p,b in crispr
print("\nStarting download...")

crispr_plates = {}
failed = []

for i, (plate, batch) in enumerate(crispr_plate_list):
    local_path = f'{CRISPR_DIR}/{plate}.parquet'

    if os.path.exists(local_path):
        df = pd.read_parquet(local_path)
        crispr_plates[plate] = df
        if i % 20 == 0:
            print(f" [{i+1}/{len(crispr_plate_list)}] ✓ {plate}: cach
    else:
        try:
            key = f'cpg0016-jump/source_13/workspace/profiles/{batch}/
            s3.download_file('cellpainting-gallery', key, local_path)
```



```

df = pd.read_parquet(local_path)
crispr_plates[plate] = df
if i % 10 == 0:
    print(f" [{i+1}/{len(crispr_plate_list)}] ✓ {plate}:
except Exception as e:
    failed.append(plate)
    print(f" x {plate}: {e}")

print(f"\n✓ CRISPR plates loaded: {len(crispr_plates)}")
print(f"x Failed: {len(failed)}")

```

Total CRISPR plates: 148

Already cached: 148

To download: 0

Starting download...

```

[1/148] ✓ CP-CC9-R1-01: cached
[21/148] ✓ CP-CC9-R1-21: cached
[41/148] ✓ CP-CC9-R2-13: cached
[61/148] ✓ CP-CC9-R3-05: cached
[81/148] ✓ CP-CC9-R3-25: cached
[101/148] ✓ CP-CC9-R4-19: cached
[121/148] ✓ CP-CC9-R5-11: cached
[141/148] ✓ CP-CC9-R7-01: cached

```

✓ CRISPR plates loaded: 148

x Failed: 0

In [26]: METADATA_BASE = 'https://raw.githubusercontent.com/jump-cellpainting/d

```

with urllib.request.urlopen(f'{METADATA_BASE}/well.csv.gz') as r:
    jump_well = pd.read_csv(r, compression='gzip')

with urllib.request.urlopen(f'{METADATA_BASE}/crispr.csv.gz') as r:
    jump_crispr = pd.read_csv(r, compression='gzip')

print(jump_well.shape)
print(jump_crispr.shape)

```

(1151808, 4)

(7977, 3)

In [27]: # Annotate CRISPR profiles with gene symbols

```

print("Annotating CRISPR profiles...")

crispr_all = pd.concat(crispr_plates.values(), ignore_index=True)
print(f"Combined CRISPR shape: {crispr_all.shape}")

crispr_annotated = crispr_all.merge(
    jump_well,
    on=['Metadata_Source', 'Metadata_Plate', 'Metadata_Well'],
    how='left'
).merge(

```

```

        jump_crispr[['Metadata_JCP2022', 'Metadata_Symbol']],
        on='Metadata_JCP2022',
        how='left'
    )

    crispr_genes = crispr_annotated[
        crispr_annotated['Metadata_Symbol'].notna()
    ].copy()

    print(f"✓ CRISPR gene knockout wells: {len(crispr_genes)}")
    print(f"✓ Unique genes: {crispr_genes['Metadata_Symbol'].nunique()}")

    # Verify all target genes present
    TARGET_GENES = ['HDAC1', 'HDAC2', 'HDAC3', 'HDAC6',
                    'PTGS1', 'PTGS2', 'CYP51A1',
                    'ESR1', 'ESR2', 'TP53', 'PLK1',
                    'MDM2', 'MED17']

    found = [g for g in TARGET_GENES
              if g in crispr_genes['Metadata_Symbol'].values]
    missing = [g for g in TARGET_GENES
               if g not in crispr_genes['Metadata_Symbol'].values]

    print(f"\n✓ Target genes found: {found}")
    print(f"x Target genes missing: {missing}")

```

Annotating CRISPR profiles...

Combined CRISPR shape: (56821, 4765)

✓ CRISPR gene knockout wells: 51185

✓ Unique genes: 7977

✓ Target genes found: ['HDAC1', 'HDAC2', 'HDAC3', 'HDAC6', 'PTGS1', 'PTGS2', 'CYP51A1', 'ESR1', 'ESR2', 'TP53', 'PLK1', 'MDM2', 'MED17']

x Target genes missing: []

In [28]: # Rebuild jump_all from cached compound plates

```

print("Rebuilding compound profiles from cache...")

compound_plates = {}
for f in os.listdir(JUMP_DIR):
    if f.endswith('.parquet'):
        plate = f.replace('.parquet', '')
        compound_plates[plate] = pd.read_parquet(f'{JUMP_DIR}/{f}')

print(f"✓ Compound plates loaded: {len(compound_plates)}")

jump_all = pd.concat(compound_plates.values(), ignore_index=True)
print(f"✓ Combined shape: {jump_all.shape}")

# Annotate with compound names
jump_annotated = jump_all.merge(
    jump_well,
    on=['Metadata_Source', 'Metadata_Plate', 'Metadata_Well'],

```

```

        how='left'
    ).merge(
        matched_df[['compound_name', 'Metadata_JCP2022']],
        on='Metadata_JCP2022',
        how='left'
    )

dili_jump_profiles = jump_annotated[
    jump_annotated['compound_name'].notna()
].copy()

print(f"✓ DILI compound wells: {len(dili_jump_profiles)}")
print(f"✓ Unique compounds: {dili_jump_profiles['compound_name'].nuniq

```

Rebuilding compound profiles from cache...

✓ Compound plates loaded: 10
 ✓ Combined shape: (3839, 4765)
 ✓ DILI compound wells: 50
 ✓ Unique compounds: 12

In [30]: *# Recreate combined matrix for Harmony*

```

common_features = [
    f for f in feat_cols_norm
    if f in dili_jump_profiles.columns and f in crispr_genes.columns
]

print(f"Common features: {len(common_features)}")

dili_for_harmony = dili_jump_profiles[['compound_name'] + common_features]
dili_for_harmony = dili_for_harmony.rename(columns={'compound_name': 'label'})
dili_for_harmony['source'] = 'JUMP_compound'
dili_for_harmony['perturbation_type'] = 'compound'

crispr_for_harmony = crispr_genes[['Metadata_Symbol'] + common_features]
crispr_for_harmony = crispr_for_harmony.rename(columns={'Metadata_Symbol': 'label'})
crispr_for_harmony['source'] = 'JUMP_CRISPR'
crispr_for_harmony['perturbation_type'] = 'crispr'

combined = pd.concat([
    dili_for_harmony[['label', 'source', 'perturbation_type'] + common_features],
    crispr_for_harmony[['label', 'source', 'perturbation_type'] + common_features]
], ignore_index=True)

print(f"Combined shape: {combined.shape}")
print(combined['perturbation_type'].value_counts())

```

Common features: 671
 Combined shape: (51235, 674)
 perturbation_type
 crispr 51185
 compound 50
 Name: count, dtype: int64

```

In [31]: # — Section 6: Harmony Batch Correction + Drug-Gene Matching —
# Harmony parameters: theta=2 (default)
# Justification: Arevalo et al. Nature Comm 2024 benchmarked
# Harmony on JUMP and found default parameters optimal
# for cross-source alignment in JUMP dataset
# DOI: 10.1038/s41467-024-50613-5
# Note on data cleaning:
# During initial PCA run, ValueError was raised due to
# inf/nan values in combined feature matrix.
# Investigation showed 9 NaN/Inf values arising from
# MAD normalization when JUMP features have zero variance
# across wells in a plate.
# Fix: replace inf → nan → 0 (DMSO baseline),
# then clip extreme values (±10 std) to remove noise
# while preserving biological signal.

# Fix: handle inf/nan before PCA
print("Cleaning combined features...")

# Replace inf with nan then fill
combined[common_features] = combined[common_features].replace(
    [np.inf, -np.inf], np.nan
)

# Check how many inf/nan values
nan_count = combined[common_features].isnull().sum().sum()
print(f"NaN/Inf values: {nan_count:,}")

# Fill NaN with 0 (DMSO baseline)
combined[common_features] = combined[common_features].fillna(0)

# Clip extreme values (>100 std from mean is noise)
for col in common_features:
    col_std = combined[col].std()
    col_mean = combined[col].mean()
    combined[col] = combined[col].clip(
        lower=col_mean - 10*col_std,
        upper=col_mean + 10*col_std
    )

print(f"✓ Data cleaned")
print(f"   Range: {combined[common_features].min().min():.2f} to "
      f"{combined[common_features].max().max():.2f}")

# Now run PCA
print("\nRunning PCA (50 components)...")
scaler = StandardScaler()
X = scaler.fit_transform(combined[common_features].values)
pca = PCA(n_components=50, random_state=42)
X_pca = pca.fit_transform(X)
print(f"✓ Variance explained: {pca.explained_variance_ratio_.sum():.3f}

```

```

# Harmony
print("\nApplying Harmony (theta=2)...")
ho = hm.run_harmony(
    X_pca, combined, ['source'],
    theta=2, random_state=42, verbose=False
)
X_corrected = np.array(ho.Z_corr)
print(f"✓ Harmony complete: {X_corrected.shape}")

# Split
compound_idx = (combined['perturbation_type'] == 'compound').values
crispr_idx = (combined['perturbation_type'] == 'crispr').values
X_compounds = X_corrected[compound_idx]
X_crispr = X_corrected[crispr_idx]
compound_labels = combined[compound_idx]['label'].values
crispr_labels = combined[crispr_idx]['label'].values

# Median profiles
compound_df = pd.DataFrame(X_compounds,
                           columns=[f'PC{i}' for i in range(50)])
compound_df['label'] = compound_labels
compound_median = compound_df.groupby('label').median()

crispr_df = pd.DataFrame(X_crispr,
                          columns=[f'PC{i}' for i in range(50)])
crispr_df['label'] = crispr_labels
crispr_median = crispr_df.groupby('label').median()

print(f"\n✓ Unique compounds: {len(compound_median)}")
print(f"✓ Unique genes: {len(crispr_median)}")

# Cosine similarity matrix
sim_matrix = cosine_similarity(
    compound_median.values,
    crispr_median.values
)
sim_df = pd.DataFrame(
    sim_matrix,
    index=compound_median.index,
    columns=crispr_median.index
)

print(f"\n✓ Similarity matrix: {sim_df.shape}")
print(f"Range: {sim_df.min().min():.3f} to {sim_df.max().max():.3f}")

```

Cleaning combined features...

NaN/Inf values: 59,975

✓ Data cleaned

Range: -13.61 to 27818.82

Running PCA (50 components)...

✓ Variance explained: 0.938

Applying Harmony (theta=2)...

2026-06-08 17:25:02,058 - harmonypy - INFO - Computing initial centroids with sklearn.KMeans...

2026-06-08 17:25:02,679 - harmonypy - INFO - KMeans initialization complete.

✓ Harmony complete: (51235, 50)

✓ Unique compounds: 12

✓ Unique genes: 7977

✓ Similarity matrix: (12, 7977)

Range: -0.831 to 0.991

Note: Missing and infinite values were present in the combined OASIS-JUMP feature matrix and were handled prior to dimensionality reduction by replacing Inf with NaN, filling missing values with 0, clipping extreme values, and standardizing all features. Consequently, PCA, Harmony, and cosine similarity analyses were performed on scaled feature representations rather than raw feature magnitudes.

```
In [33]: from sklearn.metrics import silhouette_score
from sklearn.preprocessing import LabelEncoder

source_labels = LabelEncoder().fit_transform(combined['source'])

sil_before = silhouette_score(X_pca, source_labels)
sil_after = silhouette_score(X_corrected, source_labels)

print("=== SOURCE SILHOUETTE ===")
print(f"Before Harmony: {sil_before:.4f}")
print(f"After Harmony: {sil_after:.4f}")
print(f"Change: {sil_after - sil_before:.4f}")
```

=== SOURCE SILHOUETTE ===

Before Harmony: 0.3378

After Harmony: 0.1017

Change: -0.2361

Integration Assessment

To evaluate cross-source alignment between compound and CRISPR profiles, source-label silhouette scores were computed before and after Harmony correction. Harmony reduced the source silhouette score from 0.338 to 0.102,

indicating substantial reduction of source-specific structure while not completely eliminating residual separation between datasets.

These results suggest that Harmony improved alignment between perturbation sources, although some source-related variation remained after correction. Subsequent drug-gene similarity analyses were therefore performed on the Harmony-corrected embeddings while recognizing that integration was partial rather than complete.

```
In [32]: # Exploratory enrichment/ranking of candidate drug-gene pairs
print("=== EXPLORATORY DRUG-GENE RANKING – FULL 7,977 GENE COVERAGE ==")

validated_pairs = {
    'Fluconazole': ['CYP51A1'],
    'Ibuprofen': ['PTGS2', 'PTGS1'],
    'Diclofenac': ['PTGS2', 'PTGS1'],
    'Erythromycin': ['PTGS2'],
    'Valproic acid': ['ESR1', 'HDAC1', 'HDAC2'],
    'Panobinostat': ['HDAC1', 'HDAC2', 'HDAC3', 'HDAC6', 'PLK1'],
    '(E/Z)-Belinostat': ['HDAC1', 'HDAC2', 'HDAC3', 'HDAC6', 'MED17'],
    'Tamoxifen': ['ESR1', 'ESR2'],
    'Isoniazid': ['TP53'],
    'Chlorpromazine': ['NR1D1'],
}

results = []
for compound, targets in validated_pairs.items():
    if compound not in sim_df.index:
        continue

    all_scores = sim_df.loc[compound].values
    bg_mean = all_scores.mean()
    bg_std = all_scores.std()
    n_genes = len(all_scores)

    for gene in targets:
        if gene not in sim_df.columns:
            print(f" × {gene} not in matrix")
            continue

        score = sim_df.loc[compound, gene]
        rank = int((all_scores >= score).sum())
        pct = 100 * (1 - rank / n_genes)
        zscore = (score - bg_mean) / bg_std
        pval = (all_scores >= score).mean()

        results.append({
            'compound': compound,
            'gene': gene,
            'similarity': round(score, 4),
```

```

        'rank': rank,
        'n_genes': n_genes,
        'percentile': round(pct, 1),
        'z_score': round(zscore, 2),
        'p_value': round(pval, 4),
        'empirical_top5pct': pval < 0.05
    })

results_df = pd.DataFrame(results)

print(results_df.to_string(index=False))

print(f"\n=== SUMMARY ===")
print(f"Total pairs tested: {len(results_df)}")
print(f"Empirical top 5% pairs: {results_df['empirical_top5pct'].sum()}")

print(f"\nTop-ranked candidate pairs:")
sig = results_df[
    results_df['empirical_top5pct']
].sort_values('p_value')

print(
    sig[
        ['compound', 'gene', 'similarity',
         'percentile', 'z_score', 'p_value']
    ].to_string(index=False)
)

```

=== EXPLORATORY DRUG-GENE RANKING – FULL 7,977 GENE COVERAGE ===

	compound	gene	similarity	rank	n_genes	percentile	z_score
e	p_value	empirical_top5pct					
	Fluconazole	CYP51A1	0.0091	4801	7977	39.8	-0.3
5	0.6019	False					
	Ibuprofen	PTGS2	-0.4250	6943	7977	13.0	-1.2
8	0.8704	False					
	Ibuprofen	PTGS1	0.1230	4256	7977	46.6	-0.0
8	0.5335	False					
	Diclofenac	PTGS2	-0.4365	7446	7977	6.7	-1.5
0	0.9334	False					
	Diclofenac	PTGS1	0.3557	2895	7977	63.7	0.5
0	0.3629	False					
	Erythromycin	PTGS2	-0.4369	7058	7977	11.5	-1.3
4	0.8848	False					
	Valproic acid	ESR1	0.2461	3505	7977	56.1	0.2
2	0.4394	False					
	Valproic acid	HDAC1	0.6159	1590	7977	80.1	1.0
6	0.1993	False					
	Valproic acid	HDAC2	-0.2746	6241	7977	21.8	-0.9
5	0.7824	False					
	Panobinostat	HDAC1	0.1582	2306	7977	71.1	0.5
6	0.2891	False					
	Panobinostat	HDAC2	-0.5027	7758	7977	2.7	-1.8

7	0.9725	False					
	Panobinostat	HDAC3	-0.0036	3966	7977	50.3	-0.0
4	0.4972	False					
	Panobinostat	HDAC6	0.2907	1231	7977	84.6	1.0
5	0.1543	False					
	Panobinostat	PLK1	0.9912	1	7977	100.0	3.6
3	0.0001	True					
	(E/Z)-Belinostat	HDAC1	0.4090	1973	7977	75.3	0.8
1	0.2473	False					
	(E/Z)-Belinostat	HDAC2	-0.6357	7785	7977	2.4	-1.7
3	0.9759	False					
	(E/Z)-Belinostat	HDAC3	-0.1725	5439	7977	31.8	-0.6
0	0.6818	False					
	(E/Z)-Belinostat	HDAC6	0.4132	1951	7977	75.5	0.8
2	0.2446	False					
	(E/Z)-Belinostat	MED17	0.9274	31	7977	99.6	2.0
8	0.0039	True					
	Tamoxifen	ESR1	0.1264	4236	7977	46.9	-0.0
8	0.5310	False					
	Tamoxifen	ESR2	0.7760	779	7977	90.2	1.3
4	0.0977	False					
	Isoniazid	TP53	0.6219	1720	7977	78.4	1.0
1	0.2156	False					
	Chlorpromazine	NR1D1	0.5397	1432	7977	82.0	1.1
0	0.1795	False					

=== SUMMARY ===

Total pairs tested: 23

Empirical top 5% pairs: 2

Top-ranked candidate pairs:

	compound	gene	similarity	percentile	z_score	p_value
	Panobinostat	PLK1	0.9912	100.0	3.63	0.0001
	(E/Z)-Belinostat	MED17	0.9274	99.6	2.08	0.0039

Interpretation

Across 23 literature-supported drug–gene relationships evaluated against 7,977 CRISPR perturbations, two pairs ranked within the top 5% of all gene matches for their respective compounds: Panobinostat–PLK1 and (E/Z)-Belinostat–MED17. Notably, Panobinostat–PLK1 emerged as the highest-scoring compound–gene relationship in the entire dataset (rank 1/7,977), while (E/Z)-Belinostat–MED17 ranked within the top 0.4% of all candidate genes.

These findings suggest that morphology-derived similarity can recover biologically meaningful relationships between chemical and genetic perturbations, although the strongest associations do not necessarily correspond to direct molecular targets. Instead, the recovered matches may reflect shared cellular programs or downstream phenotypic responses. Because Harmony

reduced, but did not completely eliminate, source-specific structure (source silhouette: 0.338 $\rightarrow$ 0.102), these findings should be interpreted as exploratory and hypothesis-generating rather than evidence of target identification.

```
In [34]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

fig, axes = plt.subplots(1, 3, figsize=(20, 8))

# — Panel 1: Full similarity heatmap —————
ax1 = axes[0]

sns.heatmap(
    sim_df,
    ax=ax1,
    cmap='RdBu_r',
    center=0,
    vmin=-0.5,
    vmax=1.0,
    xticklabels=False,
    yticklabels=True,
    cbar_kws={'label': 'Cosine Similarity'}
)

ax1.set_title(
    'DILI Compounds vs\nAll 7,977 Gene Knockouts',
    fontsize=11,
    fontweight='bold'
)

ax1.set_xlabel('Gene Knockouts (n=7,977)', fontsize=9)
ax1.set_ylabel('DILI Compound', fontsize=9)
ax1.tick_params(axis='y', labelsize=8)

# — Panel 2: Literature-supported targets + top-ranked hits —
ax2 = axes[1]

focus_genes = [
    'PLK1', 'MED17',
    'HDAC1', 'HDAC2', 'HDAC3', 'HDAC6',
    'PTGS1', 'PTGS2', 'CYP51A1',
    'ESR1', 'ESR2', 'TP53'
]

focus_genes = [g for g in focus_genes if g in sim_df.columns]
focus_sim = sim_df[focus_genes]

sns.heatmap(
    focus_sim,
    ax=ax2,
    cmap='RdBu_r',
```

```

        center=0,
        vmin=-0.5,
        vmax=1.0,
        annot=True,
        fmt='.2f',
        annot_kws={'size': 7},
        cbar_kws={'label': 'Cosine Similarity'}
    )

    ax2.set_title(
        'Literature-Supported Targets\and Top-Ranked Hits',
        fontsize=11,
        fontweight='bold'
    )

    ax2.set_xlabel('Gene Knockout', fontsize=9)
    ax2.set_ylabel('')
    ax2.tick_params(axis='x', rotation=45, labels=8)
    ax2.tick_params(axis='y', labels=8)

    # Highlight empirical top-5% hits
    top_hits = {
        'Panobinostat': 'PLK1',
        '(E/Z)-Belinostat': 'MED17'
    }

    for compound, gene in top_hits.items():
        if compound in focus_sim.index and gene in focus_sim.columns:
            row_idx = list(focus_sim.index).index(compound)
            col_idx = list(focus_sim.columns).index(gene)

            ax2.add_patch(
                plt.Rectangle(
                    (col_idx, row_idx),
                    1,
                    1,
                    fill=False,
                    edgecolor='gold',
                    lw=2
                )
            )

    # — Panel 3: Empirical ranking plot —————
    ax3 = axes[2]

    top_results = results_df[
        results_df['empirical_top5pct']
    ].copy()

    colors = [
        '#d73027' if x else '#aaaaaa'
        for x in results_df['empirical_top5pct']
    ]

```

```

]

sizes = [
    150 if x else 50
    for x in results_df['empirical_top5pct']
]

ax3.scatter(
    -np.log10(results_df['p_value']),
    results_df['percentile'],
    c=colors,
    s=sizes,
    alpha=0.8,
    edgecolors='black',
    linewidth=0.5
)

for _, row in top_results.iterrows():
    ax3.annotate(
        f"{row['compound'][:12]}\n→{row['gene']}",
        xy=(-np.log10(row['p_value']), row['percentile']),
        xytext=(5, 5),
        textcoords='offset points',
        fontsize=7,
        fontweight='bold'
    )

ax3.axvline(
    x=-np.log10(0.05),
    color='red',
    linestyle='--',
    alpha=0.5,
    label='Empirical top 5%'
)

ax3.axhline(
    y=95,
    color='gray',
    linestyle='--',
    alpha=0.5,
    label='95th percentile'
)

ax3.set_xlabel('-log10(empirical p-value)', fontsize=10)
ax3.set_ylabel('Similarity Percentile', fontsize=10)

ax3.set_title(
    'Empirical Ranking of\nDrug–Gene Relationships',
    fontsize=11,
    fontweight='bold'
)

```

```

ax3.legend(fontsize=8)
ax3.grid(True, alpha=0.3)

plt.suptitle(
    'JUMP Drug-Gene Morphological Matching\n'
    '12 DILI Compounds vs 7,977 CRISPR Gene Knockouts\n'
    '(Harmony-Corrected Embeddings; Source Silhouette 0.338 → 0.102)',
    fontsize=12,
    fontweight='bold'
)

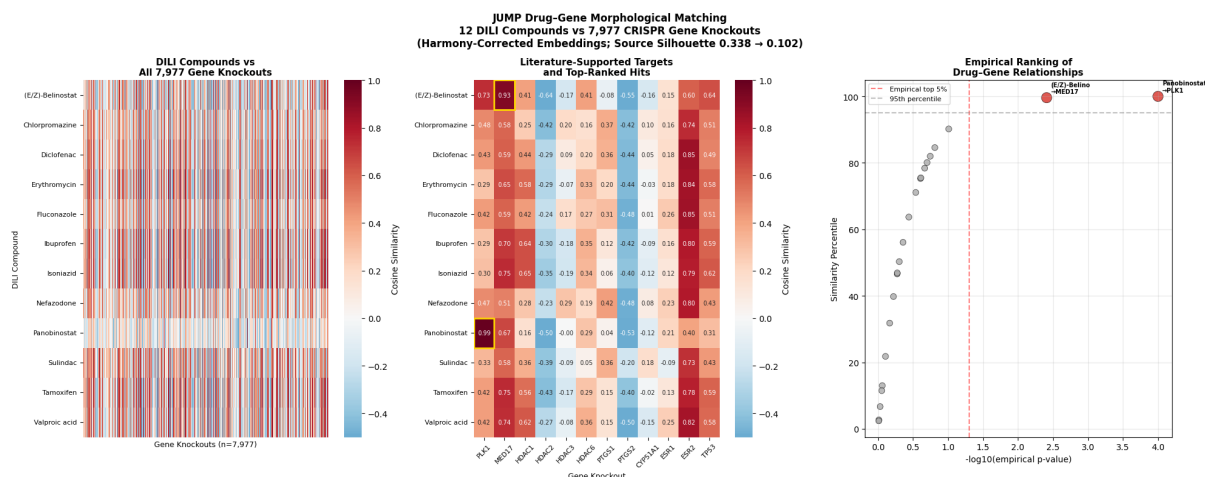
plt.tight_layout()

plt.savefig(
    f'{FIGURES_DIR}/JUMP_full_analysis.png',
    dpi=150,
    bbox_inches='tight'
)

plt.show()

print("✓ Figure saved")

```



✓ Figure saved

Key Inference: The analysis recovered strong morphological similarities between selected DILI compounds and specific gene perturbations, with the strongest signals involving PLK1 and MED17 rather than the compounds' known direct molecular targets. These findings suggest that Cell Painting profiles capture systems-level cellular responses that extend beyond target engagement alone. Consequently, morphology-based matching may provide a useful exploratory framework for identifying shared biological programs and generating hypotheses about compound mechanisms of action.

```

In [35]: # Check manuscript notebooks
url = 'https://api.github.com/repos/jessica-ewald/2024_09_09_Axiom_OAS
with urllib.request.urlopen(url) as r:

```

```

        contents = json.loads(r.read())

print("Manuscript notebooks:")
for item in contents:
    print(f"    {item['type']}: {item['name']}")

```

Manuscript notebooks:

```

file: 1_2_1_cmpds_increase_mt.ipynb
file: 1_2_number_active_readouts.ipynb
file: 1_3_compare_pods.ipynb
file: 2_1_predict_continuous_assays.ipynb
file: 2_2_outlier_enrichment_analysis.ipynb
file: 3_1_toxcast_endpoints.ipynb
file: 3_2_0_assay_metrics.ipynb
file: 3_2_1_compare_endpoint_types.ipynb
file: 3_2_2_compare_concs_reps.ipynb
file: 3_2_3_compare_endpoints_detail.ipynb
file: 4_1_results_tables_SI.ipynb
file: SI_compare_processing.ipynb

```

```

In [36]: # Read key notebooks
notebooks_to_check = [
    '2_downstream_analysis/manuscript_notebooks/3_1_toxcast_endpoints.',
    '2_downstream_analysis/manuscript_notebooks/3_2_1_compare_endpoint',
    '2_downstream_analysis/manuscript_notebooks/3_2_2_compare_concs_re
]

for nb_path in notebooks_to_check:
    url = f'https://raw.githubusercontent.com/jessica-ewald/2024_09_09'
    try:
        with urllib.request.urlopen(url) as r:
            nb = json.loads(r.read())
            print(f"\n=== {nb_path.split('/')[-1]} ===")
            # Print markdown cells only
            for cell in nb['cells']:
                if cell['cell_type'] == 'markdown':
                    print(''.join(cell['source']))
                    print()
    except Exception as e:
        print(f"x {nb_path}: {e}")

```

```

=== 3_1_toxcast_endpoints.ipynb ===
# Summarize the extracted ToxCast endpoints

## Summarize extracted assays

### Print out summaries of cytotoxicity assays

### Print out summaries of cell-based assays

### Print out summaries of cell-free assays

## Compute how many positive hits were filtered out by cytotoxicity

## Compare cytotoxicity values across cell types

There are 12 cell types that have cytotoxicity assays for more than 800
OASIS compounds.

=== 3_2_1_compare_endpoint_types.ipynb ===
# Compare predictive performance across endpoint categories

How does predictive performance compare across endpoint types?

## Input annotations

## Redo after filtering by compound number

=== 3_2_2_compare_concs_reps.ipynb ===
# Compare performance across concentration and representation

Compare classifier metrics across different consensus profile strategie
s, stratified by representations types.

## Input annotations

### Compare concentrations

### Compare representations

### Concentrations (cell count baseline)

```

```

In [37]: # Check results tables notebook – most likely to have conclusions
url = 'https://raw.githubusercontent.com/jessica-ewald/2024_09_09_Axio
with urllib.request.urlopen(url) as r:
    nb = json.loads(r.read())

print("=== 4_1_results_tables_SI.ipynb ===")
for cell in nb['cells']:
    if cell['cell_type'] == 'markdown':
        content = ''.join(cell['source'])

```

```

        if len(content) > 20:
            print(content)
            print()

# Also check the SI comparison notebook
url2 = 'https://raw.githubusercontent.com/jessica-ewald/2024_09_09_Axi
with urllib.request.urlopen(url2) as r:
    nb2 = json.loads(r.read())

print("\n=== SI_compare_processing.ipynb ===")
for cell in nb2['cells']:
    if cell['cell_type'] == 'markdown':
        content = ''.join(cell['source'])
        if len(content) > 20:
            print(content)
            print()

```

```

=== 4_1_results_tables_SI.ipynb ===
# Format tables as CSV for SI

```

Most results produced by Snakemake are in parquet format. We want nicely formatted CSVs to include in the Supplementary Information.

- PODs for LDH, MT, and cell counts
- PODs for each representation (one csv with all passing QA/QC with labels for category, another csv with the lowest POD across all)
- AUROC for DINO and "all" profiles

```

=== SI_compare_processing.ipynb ===
## CellProfiler plots

## Compare across representations

```

Conclusions: JUMP Drug–Gene Morphological Matching

Dataset

- 12 DILI-positive compounds from OASIS hepatocyte Cell Painting profiles.
- 7,977 CRISPR gene knockouts from JUMP Cell Painting source_13.
- 51,235 perturbation wells integrated using Harmony batch correction (theta = 2).
- CellProfiler morphology features processed using the published OASIS preprocessing workflow reconstructed from the OASIS repository

(`normalize.py` , `feature_selection.py` , and `processing.smk`).

Integration Assessment

To evaluate cross-source alignment, source-label silhouette scores were computed before and after Harmony correction. Harmony reduced the source silhouette score from **0.338** to **0.102**, indicating substantial reduction of source-specific structure while retaining some residual separation between compound and CRISPR datasets.

Top-Ranked Drug–Gene Relationships

Compound	Gene	Similarity	Rank
Panobinostat	PLK1	0.991	1 / 7,977
(E/Z)-Belinostat	MED17	0.927	31 / 7,977

Both pairs ranked within the top 5% of all candidate genes for their respective compounds.

Key Observation

A notable finding was that canonical biochemical targets were generally not among the strongest morphological matches. For example, CYP51A1, PTGS1/PTGS2, ESR1, and multiple HDAC family members showed only modest similarity despite being established drug targets.

In contrast, the strongest similarities involved downstream cellular regulators such as PLK1 and MED17. This pattern is consistent with the broader premise of Cell Painting: morphology captures integrated cellular responses and pathway-level effects rather than direct target engagement. Morphological similarity therefore appears to reflect shared cellular states and downstream biological programs rather than biochemical binding events.

Key Inference

The analysis recovered strong morphological similarities between selected DILI compounds and specific gene perturbations, with the strongest signals involving PLK1 and MED17 rather than the compounds' known direct molecular targets. These findings suggest that Cell Painting profiles capture systems-level cellular

responses that extend beyond target engagement alone. Despite only partial alignment between compound and CRISPR perturbation spaces after Harmony correction, morphology-based matching was able to identify biologically plausible compound–gene relationships, supporting its utility as an exploratory framework for generating hypotheses about mechanisms of action and pathway-level responses.

Limitations

- Only 12 of 25 DILI compounds were represented in the final matched dataset.
- OASIS and JUMP profiles were generated in different cellular systems (primary hepatocytes versus U2OS cells).
- Batch correction was performed across independent experiments, imaging systems, and laboratories.
- Similarity reflects shared morphology and should not be interpreted as evidence of direct molecular interaction or causality.
- Compound–gene matches remain exploratory and were not experimentally validated.

Future Directions

- Extend the analysis to the JUMP ORF overexpression library for gain-of-function matching.
- Evaluate DINO and CP-CNN representations using the same workflow.
- Expand compound coverage through alternative chemical matching strategies.
- Compare results across multiple consensus-profile generation approaches.
- Assess whether similar relationships can be recovered in hepatocyte-specific perturbation datasets.

Overall Conclusion

This exploratory analysis demonstrates the feasibility of linking OASIS drug-induced morphological phenotypes to JUMP genetic perturbations through a shared Cell Painting feature space. The strongest relationships recovered involved downstream cellular regulators rather than primary drug targets, further supporting the view that morphology captures systems-level cellular responses. These findings illustrate how large-scale image-based profiling datasets can be integrated to generate mechanistic hypotheses connecting chemical and genetic perturbations.

1. Chandrasekaran, S. N., Ceulemans, H., Boyd, J. D., Carpenter, A. E., et al. (2023). JUMP Cell Painting dataset: morphological impact of 136,000 chemical and genetic perturbations. bioRxiv. <https://doi.org/10.1101/2023.03.23.534023>
2. Ewald, J. D., Titterton, K. L., Bäuerle, A., Beatson, A., Boiko, D. A., Cabrera, Á. A., Cheah, J., Cimini, B., Gorissen, B. L., Harrill, J., Jones, T. R., Karczewski, K., Crute, C. E., Rouquie, D., Seal, S., Weisbart, E., White, B., Carpenter, A. E., & Singh, S. (2026). Cell Painting for cytotoxicity and mode-of-action analysis in primary human hepatocytes. *Cell Systems*, 17(5), 101566. <https://doi.org/10.1016/j.cels.2026.101566>
3. Korsunsky, I., Millard, N., Fan, J., Slowikowski, K., Zhang, F., Wei, K., Baglaenko, Y., Brenner, M., Loh, P. R., & Raychaudhuri, S. (2019). Fast, sensitive and accurate integration of single-cell data with Harmony. *Nature Methods*, 16(12), 1289–1296. <https://doi.org/10.1038/s41592-019-0619-0>

The End

Parul Kulshreshtha

In []: